ASSIGNMENT 1

CO1 –DSA0103

OBJECT-ORIENTED PROGRAMMING WITH C++

Submitted by: T.Nikhitha

Register Number: 192424114

1. Introduction and Assumptions

The assignment asks for an accurate, maintainable fare engine that can scale from one station (20,000 journeys/day) to a whole network (3,000,000 journeys/day). The main defects to avoid are rounding errors, ambiguous peak-hour boundaries, negative balances, and fare values repeated in several places.

1.1 Original rules

Rule	Definition used in this report
Zones	Inner = Rs 10; Middle = Rs 20; Outer = Rs 30
Peak surcharge	20% of base fare during 08:00 ≤ hour < 10 or 17:00 ≤ hour < 20
Boundary interpretation	08:00 and 17:00 are peak; 10:00 and 20:00 are off-peak
Minimum remaining balance	Balance after fare deduction must be at least Rs 50.00
Money representation	Integer paise; Rs 1.00 = 100 paise
Original concession	The brief says students and senior citizens receive a concession but does not state the percentage or exact rule. A percentage is not invented here; it must be specified before production implementation.

1.2 Goals

- Calculate every fare exactly and consistently.
- Reject invalid zone/time/balance input before changing a card.
- Keep business rules in one location so that fare revisions are manageable.
- Demonstrate correctness with boundary and invalid-input tests.
- Evaluate procedural versus object-oriented design and adapt the system to new requirements.

2. Task 1 – Understand the Problem and Identify the Objects

2.1 Objects, data and behaviour

Object / class	Data (state)	Behaviour (operations)	Reason for object
SmartCard	card ID; balance in paise; pass entitlement	canTravel(), deduct(), recharge()	Protects the balance and enforces the minimum-balance rule in one place.
Passenger	passenger ID; category such as regular/student/senior	requestJourney(); determineEligibility()	Represents a traveller and supports future concession rules.
Journey	origin/destination or farthest zone; hour; weekend flag; fare	validate(); calculateFare()	Keeps the inputs and outcome of one trip together.

Zone	zone identifier and descriptive name	provide zone metadata	Prevents unstructured text from being used as a zone value.
FareTable / FarePolicy	base fares; surcharge rate; peak periods; discounts	baseFare(); isPeak(); calculateFare()	Centralises rules and avoids duplicated numeric literals.
MonthlyPass	valid zones; validity period; passenger/card association	covers(zone); isValid(date)	Models the new unlimited-travel entitlement separately from stored-value fare deduction.

2.2 Procedural versus object-oriented approach

A procedural design stores balance, zone, hour and fare in separate variables and calls free functions such as calculateFare() and canTravel(). It can be simple for a small program, but every function that can access the balance could accidentally subtract too much or bypass validation. An object-oriented design places balance and related operations inside SmartCard and places fare rules inside FarePolicy. The class interface exposes safe operations instead of exposing internal state.

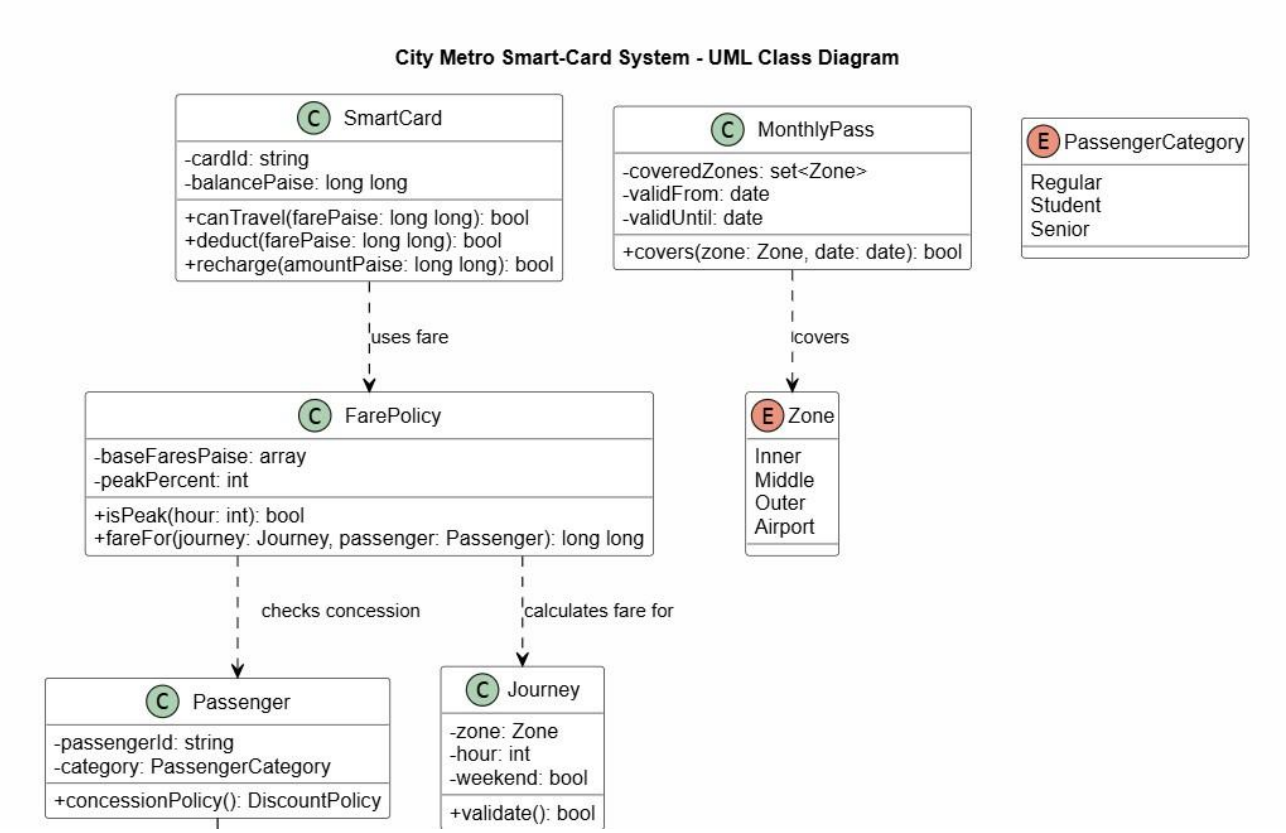
2.3 Four OOP principles in this system

- Encapsulation: SmartCard keeps balancePaise private. The only way to deduct a fare is through a validated method.
- Abstraction: callers use fareFor() and attemptTravel() without needing to know each arithmetic step.
- Inheritance: a future Passenger base class could have StudentPassenger and SeniorPassenger subclasses if their behaviour genuinely differs. Prefer composition or policy objects if only a discount rate changes.
- Polymorphism: a DiscountPolicy interface could allow StandardDiscount, StudentDiscount and SeniorDiscount implementations to calculate a concession through the same method.

2.4 Journey invariants

- Fare is non-negative and is calculated from a valid zone and hour.
- Balance is stored in paise and must never become negative.
- A journey is approved only when balance − fare ≥ 5,000 paise.
- A denied journey does not change the balance.
- Peak windows use inclusive start and exclusive end boundaries.
- All fare rates and thresholds are defined once, not repeated as unexplained literals.

2.5 UML-style class sketch



2.6 Alternative and complexity

For one journey, both procedural functions and class methods perform a fixed number of checks and arithmetic operations: O(1) time. A card stores a fixed number of fields in either design: O(1) space per card. The key difference is maintainability and rule enforcement, not asymptotic speed. A fare-rule change in procedural code may require locating several functions and shared variables; a FarePolicy class centralises the change.

3. Task 2 – Tokens, Keywords, Identifiers and Constants

3.1 Five token categories

Token category	Example	Explanation
Keyword	class, return, if, switch, constexpr	Reserved words with defined C++ meaning.
Identifier	SmartCard, balancePaise, fareFor	Programmer-chosen names; must follow identifier rules.
Constant / literal	20, 5000LL, true, 'A'	Fixed values written directly in source code.

Operator	+ , -=, >=, &&, ?:	Performs arithmetic, assignment, comparison or logical operations.
Punctuator	{ } ( ) ; , ::	Separates and structures declarations and statements.

3.2 Recommended constants

```
enum class Zone { Inner, Middle, Outer };


constexpr long long RUPEES_TO_PAISE = 100;
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
```

Use constexpr for compile-time values such as base fares and percentages. const means an object cannot be modified after initialisation, while constexpr additionally requires a constant expression when used in a context that needs compile-time evaluation. #define is a preprocessor substitution and does not provide ordinary C++ type checking or scope. enum class is safer than a plain enum because its values are scoped and do not implicitly convert to int.

Identifier conventions: use PascalCase for types (SmartCard), camelCase for functions and variables (calculateFare, balancePaise), and uppercase snake case for constants (MIN_BALANCE_PAISE). Avoid spaces, leading digits and reserved keywords.

3.3 Macro alternative and complexity

Replacing constexpr values with #define macros can leave runtime cost effectively unchanged, and neither necessarily needs a separate storage location. However, macros are not scoped typed objects and can cause surprising textual substitution. For maintainability and compiler diagnostics, scoped enums plus constexpr constants are preferred. Both fare lookup approaches remain O(1) time.

4. Task 3 – Choose Data Types and Check Type Compatibility

Value	**Recommended type**	**Reason**
Fare and card balance	long long integer paise	Exact money arithmetic; long long gives ample range for totals.
Zone	enum class Zone	Only named zone values should be accepted.
Hour	int	Small whole-number range; validate 0–23.
Peak/weekend flags	bool	Two-state conditions.
Journey count / daily totals	long long	Safe for large aggregate values.
Passenger/category character	char or enum class	char for a single character; enum class for defined categories.

Loop index	int or std::size_t	Use a type appropriate to the collection and avoid signed/unsigned warnings.
Calculated ratio for analysis	double	Suitable for non-monetary ratios; do not use as the stored fare.

Money should be stored in integer paise: Rs 10.20 becomes 1020 paise. Floating-point values such as double are binary approximations for many decimal fractions, so repeated additions can produce tiny differences. Integer paise makes fare addition and subtraction exact as long as the chosen integer type does not overflow.

4.1 Type sizes and ranges

Exact sizeof results depend on the compiler and target platform. Run the following on the submission machine and paste its output; do not assume all platforms have identical sizes.

```
#include <climits>
#include <iostream>
int main() {
    std::cout << "sizeof(bool) = " << sizeof(bool) << '\\n';
    std::cout << "sizeof(char) = " << sizeof(char) << '\\n';
    std::cout << "sizeof(int) = " << sizeof(int) << '\\n';
    std::cout << "sizeof(long long) = " << sizeof(long long) << '\\n';
    std::cout << "sizeof(float) = " << sizeof(float) << '\\n';
    std::cout << "sizeof(double) = " << sizeof(double) << '\\n';
    std::cout << "INT_MAX = " << INT_MAX << '\\n';
    std::cout << "LLONG_MAX = " << LLONG_MAX << '\\n';
}
```

If int is a 32-bit signed type, INT_MAX is 2,147,483,647. At 3,000,000 journeys, even a Rs 30 base fare produces Rs 90,000,000 (9,000,000,000 paise) before surcharge; this particular figure is below INT_MAX when measured in rupees but exceeds INT_MAX when measured in paise. A robust system should use long long for aggregate paise totals.

4.2 Double alternative and error growth

Using double rupees still gives O(1) time per addition and typically 8 bytes per value, but decimal values may be approximated. A sum of n values can accumulate rounding error depending on magnitude and order; it is not possible to state a universal error amount without running the exact experiment and knowing the values. Compare double totals and integer-paise totals in Task 10 using the same test data.

5. Task 4 – Control Implicit Conversions

Implicit conversion points include assigning an integer expression to double, promoting char to int in arithmetic, converting a non-zero integer to bool, and comparing signed and unsigned integers. Signed/unsigned comparisons can produce surprising results because a negative signed integer may be converted to a large unsigned value.

5.1 Predict-the-output program

```
int a = 7, b = 2;
double c = a / b;
double d = static_cast<double>(a) / b;
std::cout << c << ' ' << d << '\\n';
char ch = 'A' + 2;
std::cout << ch << ' ' << ch + 1 << '\\n';
bool flag = 5;
std::cout << flag << ' ' << std::boolalpha << flag << '\\n';
unsigned int u = 1;
int s = -1;
std::cout << (s < u ? "s < u" : "s >= u") << '\\n';
```

Expected reference output under ordinary C++17 implementations:

```
3 3.5
C 68
1 true
s >= u
```

Explanation: a / b is integer division because both operands are int, so 7 / 2 gives 3 before conversion to double. static_cast<double>(a) makes the division floating-point and gives 3.5. 'A' + 2 is integer arithmetic, resulting in the character C; ch + 1 promotes ch to int, so the numeric value is printed (commonly 68 for 'C' in the execution character set). A non-zero integer converts to true; boolalpha changes its printed form from 1 to true. In s < u, the signed negative value is converted for the unsigned comparison, so the comparison is false on the standard usual arithmetic conversions.

5.2 Safer conversions

```
double ratio = static_cast<double>(a) / b;
int safeCount{3};        // valid
// int invalidCount{3.7}; // compile-time error: narrowing conversion
bool enabled = (inputValue != 0);
```

Explicit casts make intent visible but should not be added blindly to every expression. Brace initialisation rejects many narrowing conversions at compile time. For valid inputs, explicit casts do not materially change asymptotic time or memory. They help expose type decisions and allow some mistakes to be caught during compilation.

6. Task 5 – Operators and Precedence

6.1 Fare expressions

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
long long surcharge = peak ? basePaise * 20 / 100 : 0;
long long finalFare = basePaise + surcharge;
```

The parentheses make the two peak windows explicit. && binds more tightly than ||, so the unparenthesised expression is logically equivalent, but the parenthesised form is easier to review. The conditional operator ?: selects one of two values after its condition is evaluated. Compound assignment such as balance -= fare updates the left operand. Keep money arithmetic in integer paise.

6.2 Increment and remainder

- x++ yields the old value and then increments x; ++x increments first and yields the new value.
- Avoid i = i++ because it has historically been a source of confusing or undefined sequencing behaviour depending on language version/context; use a separate increment statement.
- Integer division truncates toward zero. In C++17, -10 / 3 is -3 and -10 % 3 is -1 because the remainder satisfies a = (a / b) * b + a % b.

6.3 Predict-the-output program

```
int x = 5, y = 3;
std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\\n';
int z = x++ + ++y;
std::cout << z << ' ' << x << ' ' << y << '\\n';
std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\\n';
std::cout << 10 / 3 << ' ' << 10 % 3 << ' '
          << -10 / 3 << ' ' << -10 % 3 << '\\n';
int k = 0;
for (int i = 0; i < 10; i += 3) k += i;
std::cout << k << '\\n';
```

Expected reference output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication precedes addition, so 5 + (3 × 2) = 11; parentheses make (5 + 3) × 2 = 16. Line 2: x++ contributes 5 and then x becomes 6; ++y makes y 4 and contributes 4, so z = 9. Line 3: x > y is true, so the conditional expression gives 6; % and * have equal precedence and associate left to right, so (6 % 4) × 2 = 4. Line 4: integer division and remainder follow truncation toward zero. Line 5: i takes 0, 3, 6 and 9, so k = 18.

6.4 Lookup-table alternative

A bool peak[24] lookup table gives O(1) time per check and uses approximately 24 bool elements (commonly 24 bytes, but sizeof(bool) should be checked). The expression-based version is also O(1) and requires no table. Use a table if hours are configured dynamically or if the schedule is more complex; use the expression for two fixed windows.

7. Task 6 – Design the Control Structures

7.1 Pseudocode

```
ALGORITHM CalculateFare(zone, hour)
    IF hour < 0 OR hour > 23 THEN
        RETURN INVALID
    ENDIF
    SWITCH zone
        CASE INNER: baseFare ← 1000
        CASE MIDDLE: baseFare ← 2000
        CASE OUTER: baseFare ← 3000
        DEFAULT: RETURN INVALID
    ENDSWITCH
    peak ← (8 ≤ hour < 10) OR (17 ≤ hour < 20)
    IF peak THEN
        fare ← baseFare + (baseFare × 20 DIV 100)
    ELSE
        fare ← baseFare
    ENDIF
    RETURN fare
END ALGORITHM

ALGORITHM ProcessJourney(card, journey)
    fare ← CalculateFare(journey.zone, journey.hour)
    IF fare is INVALID THEN
        REPORT invalid journey
        RETURN
    ENDIF
    IF card.balancePaise – fare ≥ 5000 THEN
        card.balancePaise ← card.balancePaise – fare
        REPORT approved journey and new balance
    ELSE
        REPORT denied journey; balance unchanged
    ENDIF
END ALGORITHM
```

Use switch for a small, discrete zone enumeration; if–else for ranges and validation; and ?: for short value selection. A for loop is suitable when the number of journeys is known or stored in a collection. A range-based for loop is convenient for a collection of trips. A while loop is useful for streaming input until a sentinel or end-of-file. Use continue to skip a rejected input record and break when a required stop condition occurs. A switch default branch should reject an unknown zone.

7.2 Completed fare and travel code

```
#include <iomanip>
#include <iostream>
```

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
```

For production code, validate the enum value before calling fare() and make invalid zones return an error rather than silently using a zero base fare. The complete reference implementation in Appendix A includes this defensive validation.

7.3 Switch versus array

Both switch and array indexing are O(1) for a fixed number of zones. An array makes adding zones easy when the enum values and array order are kept aligned; a switch makes each case explicit and can provide a compiler warning when enum cases are omitted. A fourth zone requires updating the enum, fare table/switch, validation, tests, and documentation.

8. Task 7 – From Procedural to Object-Oriented

8.1 Procedural example

```
bool canTravelProcedural(long long balance, long long farePaise) {
    return farePaise >= 0 && balance - farePaise >= 5000;
}
bool deductProcedural(long long& balance, long long farePaise) {
    if (!canTravelProcedural(balance, farePaise)) return false;
    balance -= farePaise;
    return true;
}
```

8.2 Object-oriented example

```
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long minimumPaise = 5000;
public:
    explicit SmartCard(long long initialPaise)
        : balancePaise(initialPaise >= 0 ? initialPaise : 0) {}

    long long balance() const { return balancePaise; }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 &&
               balancePaise - farePaise >= minimumPaise;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }
};
```

The procedural version and class version have O(1) time per fare check and O(1) space per card. The class version better protects the invariant because other code cannot directly write a negative balance. If balance were public, any code could set it to -100 or subtract a fare without checking. When the minimum balance changes, a central constant can be updated in either design, but the class makes the rule harder to bypass.

9. Task 8 – Read and Print Data Safely (4 marks)

9.1 Input validation

```
int hour;
while (true) {
    std::cout << "Enter hour (0-23): ";
    if (std::cin >> hour) {
        if (hour >= 0 && hour <= 23) break;
        std::cout << "Hour must be from 0 to 23.\\n";
```

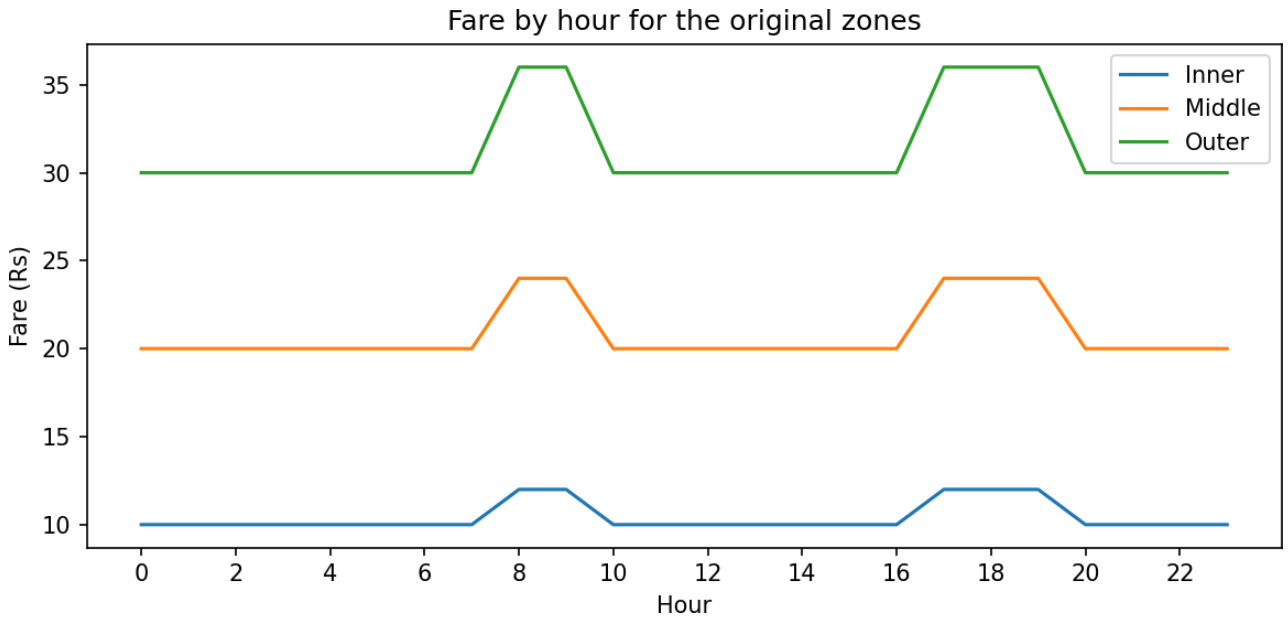
```
    } else {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
        std::cout << "Invalid input. Enter a number.\\n";
    }
}
```

cin.fail() indicates a failed formatted extraction, such as letters entered where an integer is expected. clear() resets the stream state; ignore() discards the bad line so the same characters are not read repeatedly. Include <limits> to use numeric_limits.

9.2 Money formatting

```
void printRupees(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0') << paise % 100
              << std::setfill(' ');
}
```

For line-based input, getline() followed by stoi() inside try/catch is useful when the whole input line must be validated. Both approaches process a line in O(L) time when L characters are read; getline requires a buffer proportional to the line length, O(L) space. Direct cin extraction is concise, while getline can reject trailing junk more deliberately if the full string is checked.



10. Task 9 – Implement and Test the Fare Logic

10.1 Test strategy

Tests should cover normal fares, peak and off-peak hours, exact boundary times, minimum remaining balance, invalid zone values, negative recharge, and non-numeric input. The reference table below provides expected results to compare with actual local execution.

Test	Input / condition	Expected result	Reason
T1	Inner, 07:00, balance Rs 100	Fare Rs 10.00; balance Rs 90.00	Off-peak base fare
T2	Inner, 08:00	Fare Rs 12.00	Peak begins at 08:00
T3	Middle, 09:00	Fare Rs 24.00	20% surcharge
T4	Middle, 10:00	Fare Rs 20.00	10:00 is off-peak
T5	Outer, 17:00	Fare Rs 36.00	Peak begins at 17:00
T6	Outer, 20:00	Fare Rs 30.00	20:00 is off-peak
T7	Balance Rs 60, Inner off-peak	Approved; balance exactly Rs 50	Minimum threshold is inclusive
T8	Balance Rs 59.99, Inner off-peak	Denied; balance unchanged	Would leave less than Rs 50
T9	Unknown zone	Rejected as invalid	No valid fare can be computed
T10	Recharge = -100 paise	Rejected	Recharge must be positive
T11	Hour input = abc	Error shown; re-prompt	Formatted extraction fails
T12	Fare exceeds allowable balance	Denied; no deduction	Balance invariant protected

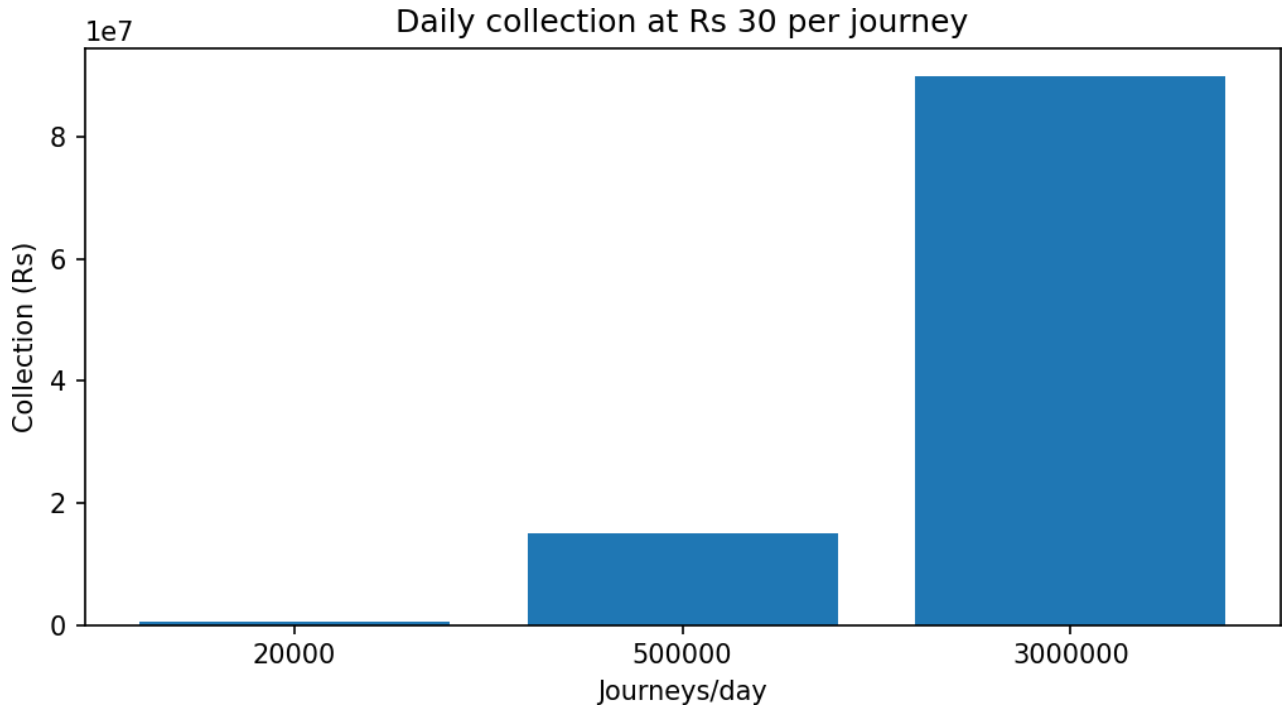
10.2 Five-journey hand calculation

Journey	Zone	Hour	Fare	Balance after trip
Start			—	Rs 100.00
1	Inner	07:00	Rs 10.00	Rs 90.00
2	Outer	18:00	Rs 36.00	Rs 54.00
3	Middle	10:00	Rs 20.00	Denied; Rs 54.00
4	Inner	08:00	Rs 12.00	Denied; Rs 54.00
5	Inner	20:00	Rs 10.00	Denied; Rs 54.00

Total deducted from the card in this example is Rs 46.00 (Rs 10 + Rs 36); the three denied journeys do not change the balance. The final balance is Rs 54.00. This hand calculation follows the assignment's minimum-balance condition.

10.3 Test table alternative and complexity

A table-driven test loop takes O(t) time for t test cases and O(t) space for the stored test table. It is easier to add a new boundary case without creating another hand-written block. Keep the expected result separate from the actual result and record PASS/FAIL after running the program.



11. Task 10 – Compare Expectations with Execution

Use the same fare list for integer-paise and double-rupee accumulation. The following is a reference scale analysis, not a substitute for measured results from the student's compiler.

Journeys/day	At Rs 10 each	At Rs 20 each	At Rs 30 each	Maximum with 20% peak surcharge on all Rs 30 trips
20,000	Rs 200,000	Rs 400,000	Rs 600,000	Rs 720,000
500,000	Rs 5,000,000	Rs 10,000,000	Rs 15,000,000	Rs 18,000,000
3,000,000	Rs 30,000,000	Rs 60,000,000	Rs 90,000,000	Rs 108,000,000

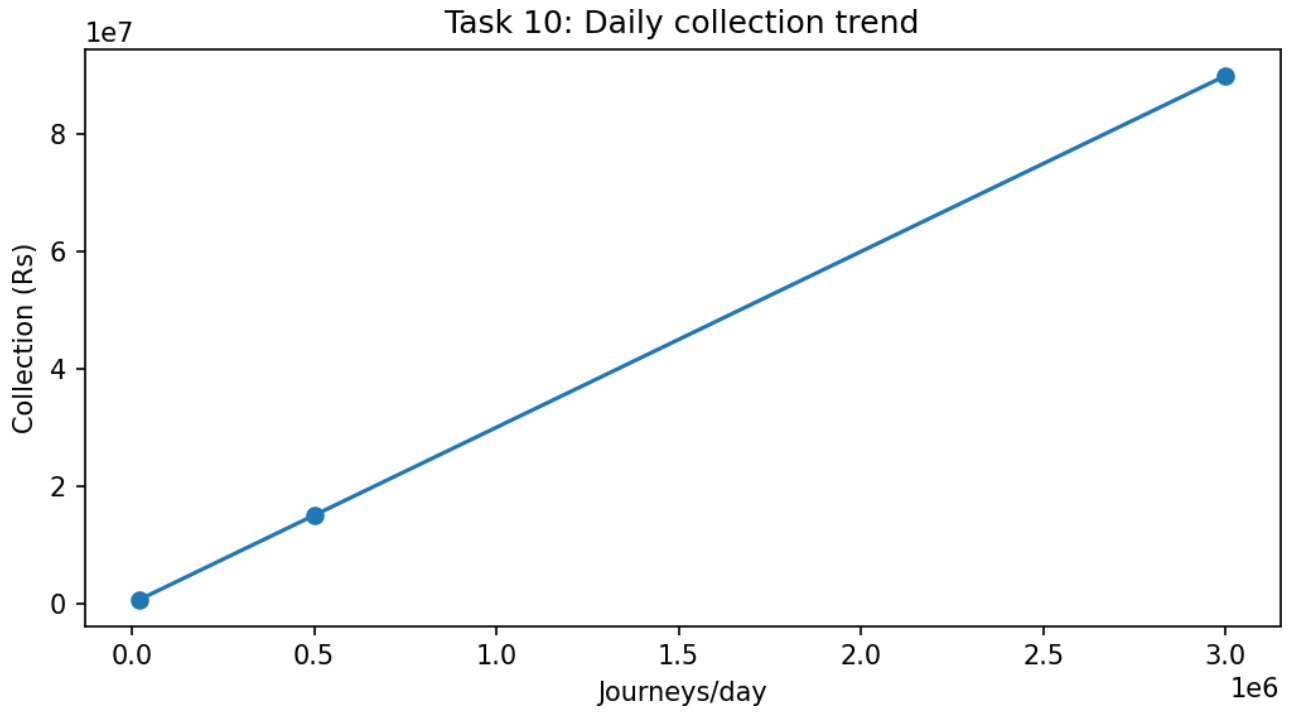
At the whole-network scale, Rs 108,000,000 equals 10,800,000,000 paise. A 32-bit signed int cannot store that paise total, although it can store the same amount measured in rupees. This demonstrates why the unit of the stored value matters as much as the data type.

11.1 Measurement procedure

```
// Record actual totals using the same fares in both representations:
long long totalPaise = 0;
double totalRupees = 0.0;
for (long long f : faresPaise) {
    totalPaise += f;
    totalRupees += static_cast<double>(f) / 100.0;
}
std::cout << "Integer total (paise): " << totalPaise << '\\n';
std::cout << std::fixed << std::setprecision(2)
          << "Double total (Rs): " << totalRupees << '\\n';
```

Expected mathematical totals should agree after converting paise to rupees, but double may differ by a tiny rounding amount for suitable input sequences. Do not claim a measured difference until the experiment has been run. With -Wall -Wextra, possible warnings include signed/unsigned comparison, unused variables, missing return paths, or implicit narrowing, depending on the submitted code. Copy the exact warnings from the local compiler and explain each one.

At 3,000,000 additions, int addition and long long addition are O(1) per operation; the aggregate loop is O(n). A single int or long long variable is O(1) space. double is also O(1) space, but may lose exact decimal representation. Integer overflow of a signed type is undefined behaviour in C++; do not rely on wraparound.



12. Task 11 – Recommend and Defend the Final Design

12.1 Recommended design

- Store fares, balances and totals as long long integer paise.
- Use enum class Zone for zone values and constexpr constants for the original fare rules.
- Use SmartCard with private balance and validated deduct()/recharge() operations.
- Put peak-window and fare calculations in one FarePolicy component.
- Validate all user input and reject unknown zones or invalid hours before processing.
- Use table-driven tests and record actual compiler outputs, warnings and screenshots.
- Keep journey processing linear in the number of journeys; each fare calculation is O(1), so n journeys take O(n) time.

12.2 Revision procedure

For a fare revision, change the central fare table or configuration and update the related tests. Avoid editing repeated numeric literals in several functions. The design is suitable for millions of journeys because each journey can be processed in constant time and the complete day in O(n) time. If all journeys are stored in memory, storage is O(n); if they are processed as a stream, working memory can remain O(1), excluding logs.

12.3 Limitations

- The original brief does not specify the student/senior concession percentage, rounding policy, or whether concession and peak surcharge combine. Those rules must be clarified.
- The example does not model refunds, concurrency, database failures, card security, payment settlement or audit logging.
- The monthly pass requires validity dates and covered-zone rules; it should not be forced into a single boolean flag on SmartCard.

12.4 Configuration-file alternative

Reading z fares from a configuration file requires O(z) startup time and O(z) storage for the fare table, after which each lookup remains O(1). It avoids recompiling for fare-only changes, but requires file validation, versioning, safe defaults and failure handling. For a small educational program, constexpr is simpler; for an operational metro with frequent tariff changes, a validated configuration service may be preferable.

13. Task 12 – Day-4 Design Challenge (8 marks)

13.1 A. Impact analysis (2 marks)

Existing component	Required change	Reason

Zone enum	Add Airport	Fourth zone is a new valid destination zone.
Fare table / switch	Add Airport base fare = 6000 paise	Rs 60.00 base fare.
Fare policy	Add weekend discount = 10%	Discount applies on weekends and does not combine with peak surcharge.
Journey	Add weekend flag or date	Fare policy needs to know whether the trip is on a weekend.
SmartCard	Add pass entitlement reference or query pass separately	Card balance and pass entitlement are different concepts.
MonthlyPass class	Add two covered zones and validity period	Unlimited travel applies only to two chosen zones during validity.
Tests	Add airport, weekend, peak/weekend interaction and pass tests	New rules introduce new boundaries and interactions.

13.2 B. Evaluate original design (2 marks)

The original switch can be retained for four zones, but every additional zone requires another case and coordinated updates. A table indexed by a validated enum is more convenient when zones are data-driven. SmartCard remains useful for stored-value travel, but monthly pass eligibility should be represented by a separate MonthlyPass object. The discount should be an explicit policy decision: when weekend is true, apply the 10% weekend discount and do not add the peak surcharge.

Worked examples, assuming the discount is applied to the base fare and integer paise arithmetic truncates fractional paise: Airport on a weekday peak hour = Rs 60 + 20% = Rs 72.00. Airport on a weekend peak hour = Rs 60 − 10% = Rs 54.00, with no peak surcharge. Inner on a weekend = Rs 9.00. These examples follow the stated non-combination rule; the final policy should confirm the rounding rule.

13.3 C. Revised design and code (2 marks)

```
#include <array>
#include <optional>

enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long, 4> BASE_FARE = {
    1000, 2000, 3000, 6000
};
constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

bool isPeak(int hour) {
    return hour >= 8 && hour < 10 ||
           hour >= 17 && hour < 20;
}
```

```
std::optional<long long> calculateFare(
    Zone zone, int hour, bool weekend) {
    if (hour < 0 || hour > 23) return std::nullopt;
    auto index = static_cast<std::size_t>(zone);
    if (index >= BASE_FARE.size()) return std::nullopt;

    long long base = BASE_FARE[index];
    if (weekend) {
        // Weekend discount replaces, and does not combine with, peak surcharge.
        return base - (base * WEEKEND_DISCOUNT_PERCENT / 100);
    }
    if (isPeak(hour)) {
        return base + (base * PEAK_PERCENT / 100);
    }
    return base;
}

class MonthlyPass {
private:
    Zone firstZone;
    Zone secondZone;
    int validFromDay;
    int validUntilDay;
public:
    MonthlyPass(Zone a, Zone b, int from, int until)
        : firstZone(a), secondZone(b),
          validFromDay(from), validUntilDay(until) {}

    bool covers(Zone z, int day) const {
        bool validDate = day >= validFromDay && day <= validUntilDay;
        bool coveredZone = z == firstZone || z == secondZone;
        return validDate && coveredZone;
    }
};
```

This code is a compact teaching model. A production implementation should validate enum values without assuming arbitrary enum casts are valid, represent dates with a real date type, prevent duplicate covered zones if that is disallowed, and define how a valid pass interacts with stored-value balances.

13.4 D. Evidence-based defence (2 marks)

Test	Expected fare / outcome	Design point validated
Inner, weekday 08:00	Rs 12.00	Peak surcharge applies at the start boundary.
Inner, weekday 10:00	Rs 10.00	Peak surcharge ends at 10:00.

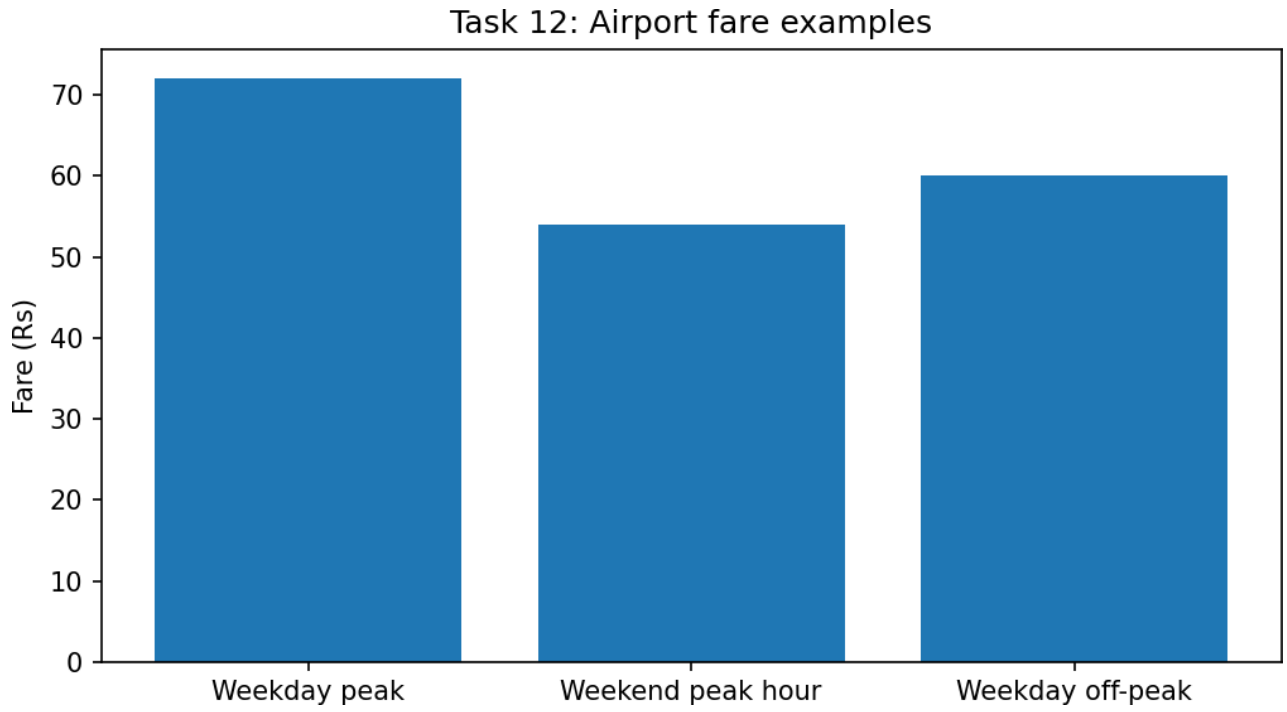
Airport, weekday peak	Rs 72.00	New zone and peak calculation work together.
Airport, weekend peak	Rs 54.00	Weekend discount replaces peak surcharge.
Inner, weekend off-peak	Rs 9.00	10% weekend discount applies.
Monthly pass covers zone and date	Pass travel; no stored fare charged under pass policy	Pass entitlement is separate from fare calculation.
Monthly pass does not cover zone/date	Normal fare calculation	Pass scope and validity are checked.

Decision: MODIFY the original design rather than completely redesigning it. Retain SmartCard encapsulation and the integer-paise approach; modify the fare policy to use a validated fare table and an explicit weekend rule; add MonthlyPass as a separate class. This limits changes to the fare-policy and entitlement components. The final defence must be supported with the student's executed test output and screenshots.

14. Overall Complexity, Limitations and Conclusion

Operation	**Time complexity**	**Space complexity**
Single fare calculation	O(1)	O(1) extra
Single balance eligibility check	O(1)	O(1)
Process n journeys sequentially	O(n)	O(1) working space, excluding logs
Run t table-driven tests	O(t)	O(t) for stored test cases
Load z fares from configuration	O(z) startup; O(1) lookup	O(z) fare table
Check MonthlyPass coverage	O(1) for two stored zones	O(1)

The recommended solution uses exact integer-paise arithmetic, named constants, scoped zone values, explicit boundary checks, and a private card balance. These choices address the original failure modes and keep fare calculations efficient as the network grows. The extended design supports the Airport zone, a non-combinable weekend discount, and monthly passes without mixing unrelated responsibilities.



Github link : https://github.com/nikitha765432/T.Nikhitha-192424114-DSA0103-C-